

Prática DevOps: Containerização Social Ledger

Disciplina: DevOps

Autor: João Paulo Morais Rangel

1. Visão Geral da Aplicação

O **Social Ledger** é uma ferramenta full-stack de simulação financeira projetada para simplificar a gestão de despesas compartilhadas entre grupos de pessoas. A aplicação permite que os usuários criem grupos, registrem participantes (incluindo convidados simulados), lancem gastos e visualizem o fechamento de contas.

O diferencial técnico da ferramenta é o seu algoritmo de liquidação de dívidas implementado no backend. Utilizando uma abordagem de grafos com *Priority Queues* (Filas de Prioridade), o sistema calcula o número mínimo de transações necessárias para zerar todos os débitos do grupo, resolvendo de forma eficiente o problema de compensação "N-para-N".

2. Tecnologias Utilizadas

Camada	Tecnologia	Descrição / Papel
Backend	Java 21 / Spring Boot 4	API RESTful com arquitetura limpa (Clean Architecture).
Frontend	React / TypeScript / Vite	Interface Single Page Application (SPA) responsiva.
Banco de Dados	PostgreSQL 15	Persistência de dados relacional e transacional.
Servidor Web	Nginx	

Camada	Tecnologia	Descrição / Papel
		Hospedagem do frontend e Proxy Reverso para a API.
Orquestração	Docker & Docker Compose	Gerenciamento do ciclo de vida dos contêineres e redes.

3. Arquitetura de Contêineres (DevOps)

Para atender aos requisitos da disciplina, a aplicação foi estruturada em três serviços independentes que se comunicam através de uma rede interna isolada:

3.1. Contêiner de Banco de Dados (db)

- **Imagem:** `postgres:15-alpine`.
- **Persistência:** Utiliza um volume nomeado (`postgres_data`) para garantir que os dados não sejam perdidos ao reiniciar os contêineres.

3.2. Contêiner de API (api)

- **Construção:** Utiliza *Multi-stage build*. O primeiro estágio usa Maven para compilar o código Java 21, e o segundo utiliza um JRE leve para execução.
- **Conectividade:** Conecta-se ao banco de dados via rede interna usando o DNS do Docker (`db:5432`).

3.3. Contêiner de Frontend (frontend)

- **Papel:** Atua como servidor web de arquivos estáticos e **Proxy Reverso**.
- **Configuração:** O Nginx intercepta requisições enviadas para `/api` e as redireciona internamente para o contêiner `api`. Isso elimina problemas de CORS no navegador e centraliza o acesso na porta 80.

4. Manual de Instalação e Execução

Graças à containerização, a aplicação pode ser executada em qualquer ambiente que possua o Docker instalado, sem a necessidade de configurar Java, Node.js ou PostgreSQL localmente.

Pré-requisitos

- Docker Desktop ou Docker Engine instalado.
- Docker Compose configurado.

Passos para Execução

1. Navegue até a raiz do projeto onde se encontra o arquivo `docker-compose.yml`.
2. Execute o comando para construir e iniciar os serviços:

```
docker-compose up -d --build
```

3. Aguarde a inicialização dos serviços (o banco de dados e a API levam alguns segundos para estarem prontos).
4. Acesse a aplicação no navegador em: <http://localhost>

Comandos Úteis

- Verificar logs: `docker-compose logs -f`
- Parar aplicação: `docker-compose down`
- Limpar volumes (apagar dados): `docker-compose down -v`